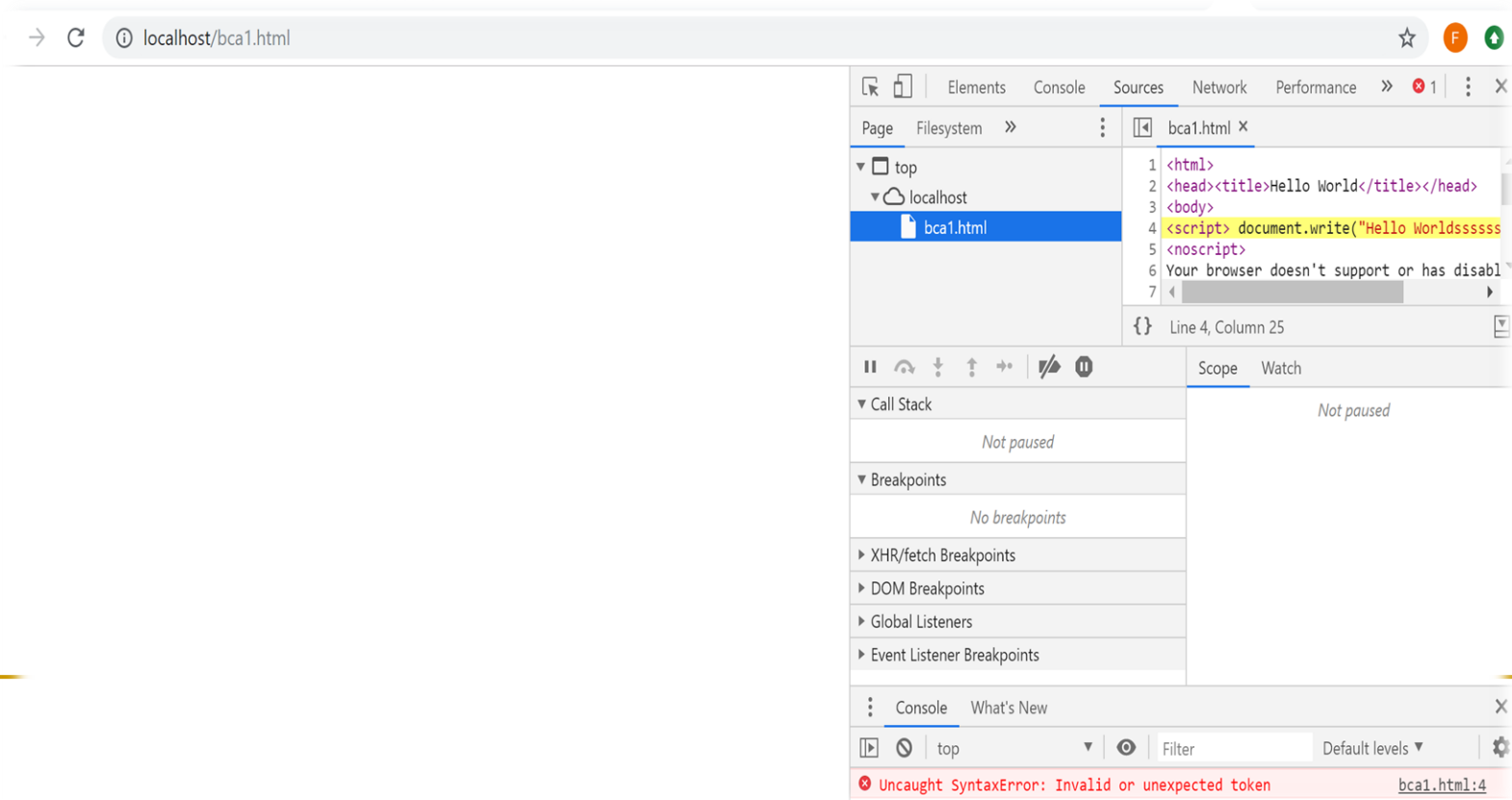

JavaScript Tutorial

Ref: <https://www.w3schools.com/> and
other internet based tutorials

Accessing JavaScript error messages in Google Chrome browsers

- When learning JavaScript, it's important to be able to track typing or other coding errors. Use following options in Google Chrome:
Right click on the browser and click inspect



JavaScript Syntax

■ Using Comments

- Single-line comment, like : `// This is a comment`
- Multiline comments, like this:

/ This is a section
of multiline comments
that will not be
interpreted */*

■ Semicolons

- JavaScript generally does not require semicolons if we have only one statement on a line. However, when we wish to place more than one statement on a line, it must be separated with semicolons.

■ Variables

- Variables use the following naming rules:
- A variable may include only the letters a-z, A-Z, 0-9, the \$ symbol, and the underscore (_).
- No other characters, such as spaces or punctuation, are allowed in a variable name.
- The first character of a variable name can be only a-z, A-Z, \$, or _ (no numbers).
- Names are case-sensitive. Count, count, and COUNT are all different variables.
- There is no set limit on variable name lengths.
- JavaScript string variables should be enclosed in either single or double quotation marks

■ Variable Typing

- JavaScript is a very loosely typed language; the type of a variable is determined only when a value is assigned and can change as the variable appears in different contexts.

■ String Concatenation

- JavaScript handles string concatenation using the plus sign (+), like this: `document.write("You have " + messages + " messages.")`.
- Just as we can add a value to a numeric variable with the `+=` operator, we can also append one string to another the same way:

```
name = "James"
```

```
name += " Dean"
```

Document Object Model(DoM)

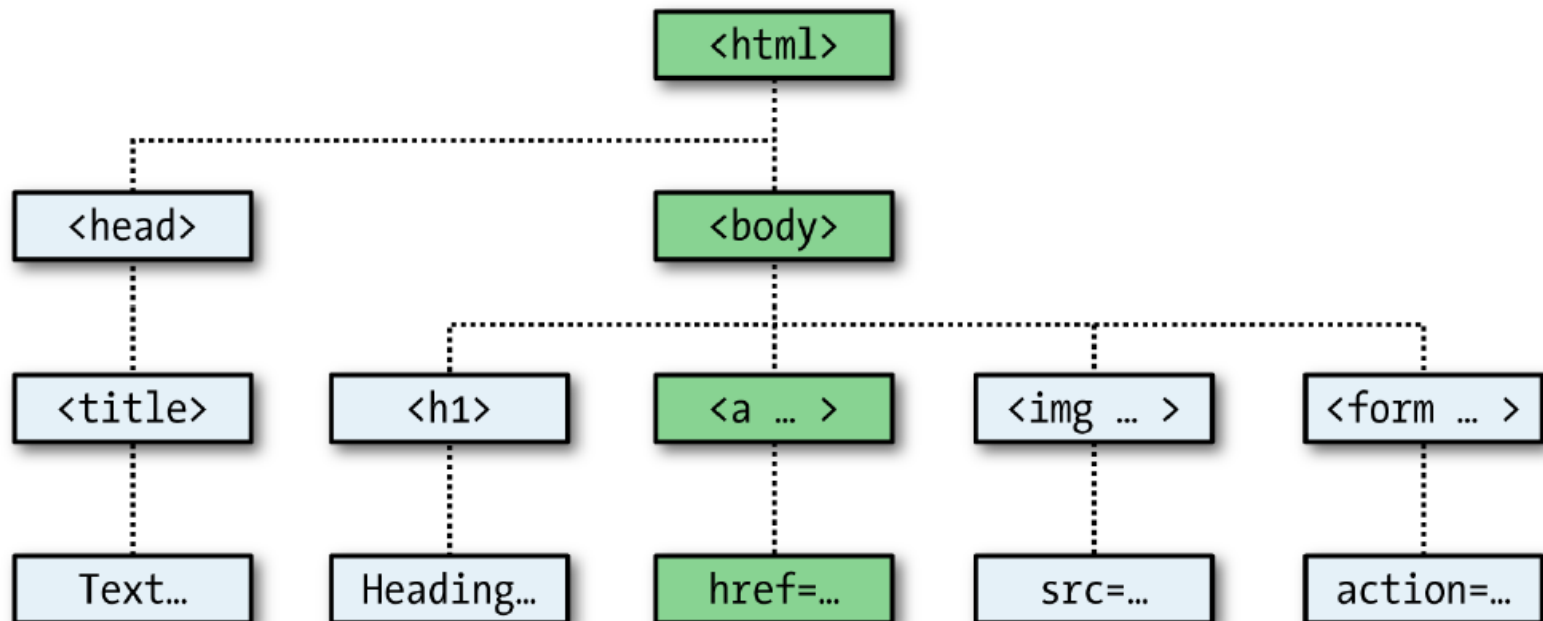
- Within JavaScript, there is a hierarchy of parent and child objects, which is what is known as the Document Object Model.
- This breaks down the parts of an HTML document into discrete *objects*, each with its own *properties* and *methods* and each subject to JavaScript's control.
- For example, let's consider a employee as an object. This object contains properties such as a name, address, phone number, and so on. In the syntax of JavaScript, these properties would look like this:
 employee.name
 employee.phone
 employee.address
- Its methods are functions that retrieve, change, the properties. For instance, to invoke a method that displays the properties of employee, you might use syntax such as this: employee.display()

Document Object Model(DoM) cont...

- The figure uses HTML tags that illustrate the parent/child relationship between the various objects in a document. For example, a URL within a link is part of the body of an HTML document. In JavaScript, it is referenced like this:

```
url = document.links.linkname.href
```

The first part, document, refers to the `<html>` and `<body>` tags; `links.linkname` to the `<a>` tag; and `href` to the `href` attribute.



Example of DOM

```
<html>
```

```
<head>
```

```
<title>Link Test</title>
```

```
</head>
```

```
<body>
```

```
<a id="mylink" href="http://mysite.com">Click me</a><br>
```

```
<script>
```

```
url = document.links.mylink.href
```

```
document.write('The URL is ' + url)
```

```
</script>
```

```
</body>
```

```
</html>
```

Using the DOM

- The links object is actually an array of URLs, so the URLs can also be safely referred to on all browsers in the following way :

```
url = document.links[i].href
```

- If you want to know how many links there are in an entire document, you can query the length property of the links object like this:

```
numlinks = document.links.length
```

- You can therefore extract and display all links in a document like this:

```
for (j=0 ; j < document.links.length ; ++j)  
document.write(document.links[j].href + '<br>')
```

Expressions

- JavaScript expression is a combination of values, variables, operators, and functions that results in a value; the result can be a number, a string, or a Boolean value (which evaluates to either true or false).

```
<script>
```

```
document.write("a: " + (42 > 3) + "<br>")
```

```
document.write("c: " + (8 == 2) + "<br>")
```

```
</script>
```

- In JavaScript true and false are referenced in lowercase. Therefore, only the first of the two following statements will display:

```
if (1 == true) document.write('true') // True
```

```
if (1 == TRUE) document.write('TRUE') // Will cause an error
```

Operators

- JavaScript offers a lot of powerful operators that range from arithmetic, string, and logical operators to assignment, comparison, and more

Operator	Description	Example
Arithmetic	Basic mathematics	<code>a + b</code>
Array	Array manipulation	<code>a + b</code>
Assignment	Assign values	<code>a = b + 23</code>
Bitwise	Manipulate bits within bytes	<code>12 ^ 9</code>
Comparison	Compare two values	<code>a < b</code>
Increment/decrement	Add or subtract one	<code>a++</code>
Logical	Boolean	<code>a && b</code>
String	Concatenation	<code>a + 'string'</code>

■ The with Statement

- References to properties and methods within the with block are assumed to apply to the object specified in with statement.

```
<script>
```

```
string = "The quick brown fox jumps over the lazy dog"
```

```
with (string)
```

```
{
```

```
document.write("The string is " + length + " characters<br>")
```

```
document.write("In upper case it's: " + toUpperCase())
```

```
}
```

```
</script>
```

- Even though string is never directly referenced by document.write, this code still manages to output the following:

- **The string is 43 characters**

- **In upper case it's: THE QUICK BROWN FOX JUMPS OVER
THE LAZY DOG**

Conditionals

- There are three types of nonlooping conditionals: the if statement, the switch statement, and the ? operator.

- The if else Statement

```
if (a > 100)
{
document.write("a is greater than 100")
}
else if(a < 100)
{
document.write("a is less than 100")
}
else
{
document.write("a is equal to 100")
}
```

■ The switch Statement

```
var x = "0";  
switch (x) {  
  case 0:  
    text = "Off";  
    break;  
  case 1:  
    text = "On";  
    break;  
  default:  
    text = "No value found";  
}
```

The ? Operator

- The ternary operator (?), combined with the : character, provides a quick way of doing if...else tests

<script>

document.write(

a <= 5 ? "a is less than or equal to 5" : "a is greater than 5"

)

</script>